MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

UNIVERSITY Of KIGALI

K means Clustering

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression, and pattern discovery
- Useful when you need structure from raw, unorganized data

K-means is a popular unsupervised machine learning algorithm used for clustering data. Its main goal is to group a set of data points into K distinct clusters based on their features, where K is a number you choose beforehand. explain this definition

- **K-means is a popular unsupervised machine learning algorithm used for clustering data."** This means K-means is a widely used method in machine learning that doesn't require labeled data (unsupervised). Its purpose is to find groups or clusters within a dataset by analyzing the data itself.

- **Its main goal is to group a set of data points into K distinct clusters based on their features,**
  The algorithm organizes data points into K separate groups, where each group contains points that are similar to each other based on their characteristics or measured attributes (features).

- **where K is a number you choose beforehand.**
  Before running the algorithm, you decide how many clusters (K) you want the data to be divided into. This number controls how many groups the algorithm will create.

In summary, K-means helps you discover groups in your data by dividing it into a specified number of clusters, grouping similar points together based on their properties. Let me know if you'd like me to clarify further or provide examples!

What does K-means do?

- **It partitions the data into K clusters:**
  The algorithm starts by deciding to divide the entire dataset into a specific number of groups, called clusters. The number of clusters, K, is chosen before running the algorithm based on what you want to find or understand in your data.
- **Each cluster has a centroid (a center point), which is the average of all points in that cluster:**
  Each cluster is represented by a centroid a point that serves as the "center" of the cluster. This centroid is calculated by taking the average (mean) of all the data points that belong to that cluster

UNIVERSITY Of KIGALI

- **The algorithm assigns each data point to the cluster whose centroid is closest to it:**
  After initializing centroids, the algorithm looks at each data point and measures its distance to each centroid. It then assigns the point to the cluster with the nearest centroid, effectively grouping points that are similar or close together in the feature space

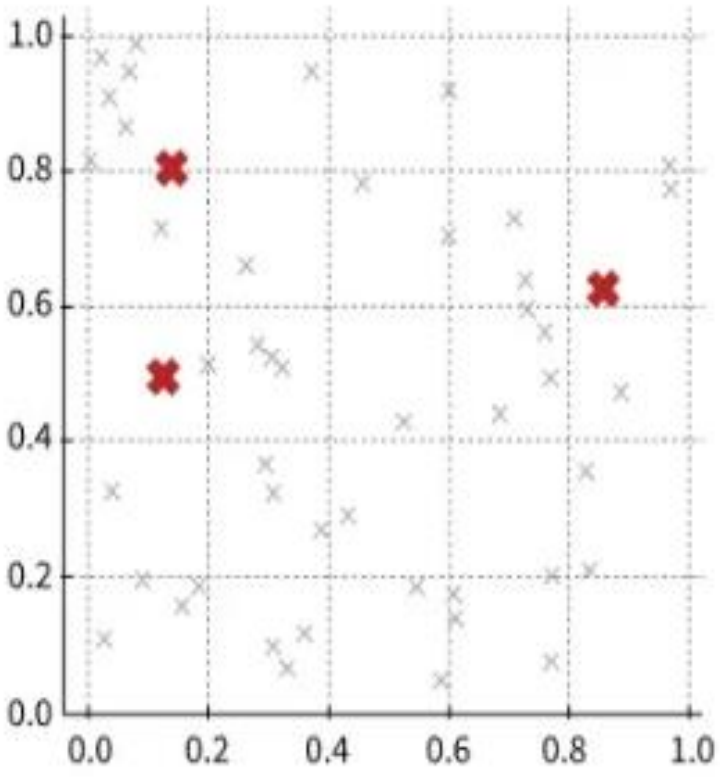
- **It iteratively updates the centroids and reassigns points until the clusters stabilize:**
  Once all points are assigned, the centroids are recalculated based on the new cluster memberships. This process of assigning points to clusters and updating centroids repeats multiple times.

The algorithm continues until the centroids no longer move significantly or the assignments of points to clusters stop changing, meaning the clusters have stabilized or converged

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

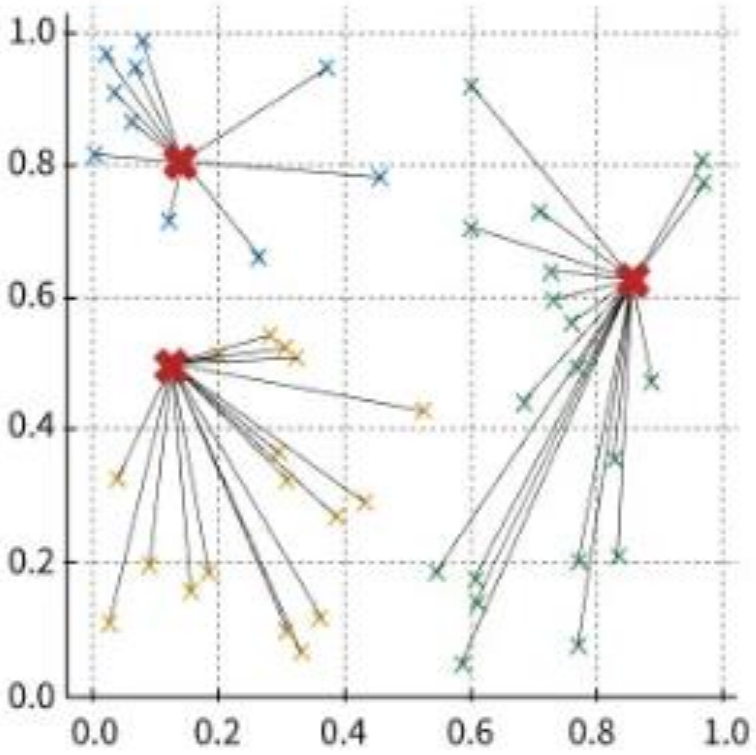
✖ Centroids × Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

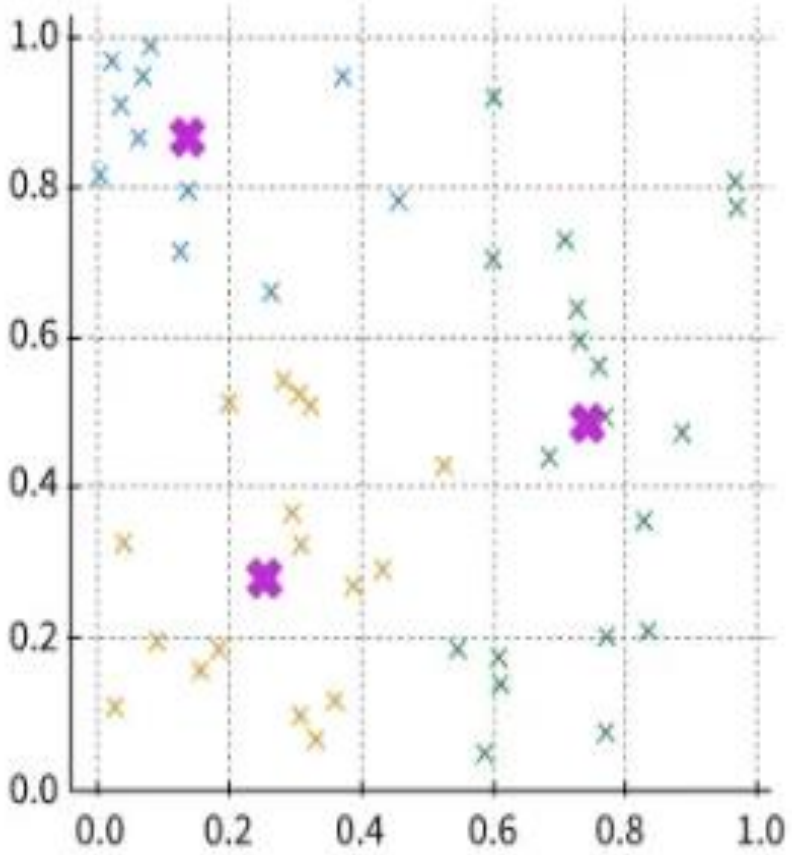
- Centroids
- Cluster 1
- Cluster 2
- Cluster 3



Update Centroids

Centroids are recalculated as the mean of the points in each cluster

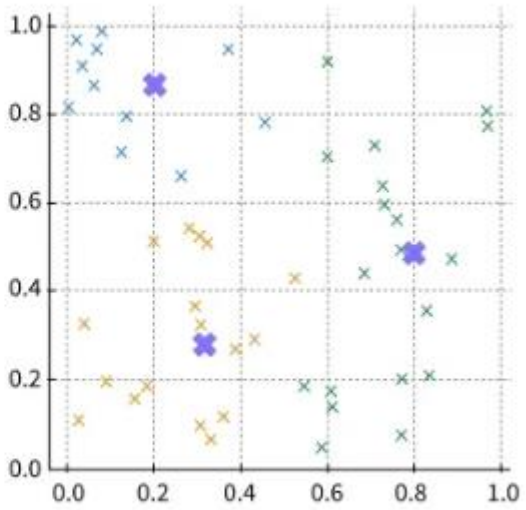
- New Centroids
- Cluster 1
- Cluster 2
- Cluster 3



Repeat Until Convergence

This process repeats until the centroids stabilize and do not move further.

- Final Centroids
- Cluster 1
- Cluster 2
- Cluster 3



Why use K-means?

- **It helps identify patterns or groups in data without prior labels:**
  K-means is an unsupervised learning technique, meaning it does not require pre-labeled data to work. This makes it very useful when you have a large set of raw data and want to discover hidden structures or natural groupings within it. By automatically grouping similar data points together, K-means reveals patterns that might not be obvious just by looking at the data.

UNIVERSITY Of KIGALI

Useful in market segmentation:

Businesses use K-means to divide their customers into distinct segments based on purchasing behavior, demographics, or preferences.

This helps companies tailor marketing strategies and product offerings to different customer groups, improving customer satisfaction and sales effectiveness.

Useful in image compression:

K-means can reduce the number of colors in an image by clustering similar colors together and representing each cluster by its centroid color. This reduces the image file size while preserving visual quality, making it efficient for storage and transmission.

Useful in document clustering:

It groups similar documents, articles, or texts into clusters based on their content. This helps in organizing large collections of documents, improving search results, and enabling topic discovery in text mining applications.

Implementation Steps of K-Means Clustering with General Syntax:

1.Import Necessary Libraries

When implementing K-means clustering, you typically need to import some key libraries to handle data and perform the clustering.

These libraries provide ready-made functions and tools that make the process simpler and more efficient

Common libraries to import:

- **NumPy (numpy):**

- Used for numerical operations and handling data in array form.
- It provides support for multi-dimensional arrays, mathematical functions, and more.
- Example: You can store your dataset as a NumPy array for easy manipulation.

- **scikit-learn (sklearn):**

- A popular machine learning library in Python.
- Contains the KMeans class which implements the K-means clustering algorithm.
- This saves you from coding the algorithm from scratch, allowing you to quickly apply K-means to your data.

UNIVERSITY of KIGALI

Matplotlib (matplotlib): (optional but useful)

- A plotting library for creating visualizations.
- You can use it to visualize clusters and centroids, which helps understand how the algorithm grouped your data.

Example: Importing Libraries in Python

```
# For numerical operations and handling arrays import numpy as np
# For K-means clustering algorithm
from sklearn.cluster import KMeans
# For plotting and visualization (optional) import matplotlib.pyplot as plt
```

2. Prepare Your Dataset

Before applying the K-means algorithm, you need to have your data ready.

This involves either loading existing data from a file or defining your own dataset within the code. Your dataset should be in a format that the algorithm can process usually a numerical matrix where rows represent individual data points and columns represent features or attributes.

Example: Loading Dataset from a CSV File

```
import pandas as pd
# Load data from a CSV file
df = pd.read_csv('your_data.csv')
# Convert DataFrame to NumPy array for clustering
data = df.values
```

Initialize the K-Means Model

After preparing your dataset, the next step is to create and configure the K-Means clustering model before running it on your data.

- Key aspects of initializing the K-Means model:

Specify the number of clusters (K):

- ✓ This is the most important parameter. It tells the algorithm how many groups to form from the data.
- ✓ Choosing the right K depends on your data and the problem you are solving. Sometimes you try different K values and compare results using metrics like the elbow method or silhouette score.

UNIVERSITY Of KIGALI

Set the random state (optional but recommended):

- K-Means initializes cluster centroids randomly by default, which can lead to different results on each run.
- Setting a random seed (random_state) ensures reproducibility—your results will be the same every time you run the code.

Specify the maximum number of iterations:

- Limits how many times the algorithm will repeat the process of assigning points and updating centroids before stopping.
- Default is often sufficient, but you can increase it if your data needs more time to converge.

- **Choose the initialization method:**

- The method for choosing initial centroids can affect performance and speed.
- Common options include 'k-means++' (smart initialization to speed up convergence) and 'random'

UNIVERSITY Of KIGALI

Other optional parameters:

- Tolerance level for convergence (how much centroid movement is allowed before stopping).
- Whether to run the algorithm multiple times with different centroid seeds (n_init) to pick the best result.

Example: Initializing the K-Means Model in Python

```
from sklearn.cluster import KMeans

# Create KMeans instance with 3 clusters
kmeans = KMeans(
    n_clusters=3,          # Number of clusters
    init='k-means++',      # Centroid initialization method
    max_iter=300,          # Maximum iterations per run
    n_init=10,             # Number of initializations to run
    random_state=42        # Seed for reproducibility
)
```

Fit the Model to the Data

Once you have initialized your K-means model and prepared your dataset, the next step is to run the algorithm using your data. This process is called "fitting" the model.

What happens during fitting?

1. Assignment of points to clusters:

The algorithm calculates the distance between each data point and the centroids (initially randomly chosen or initialized in a smart way). Each point is assigned to the cluster with the nearest centroid.

- **Centroid update:**

After all points are assigned, the algorithm recalculates the centroid of each cluster by taking the average of all points currently assigned to that cluster.

- **Iteration:**

Steps 1 and 2 are repeated reassigning points and updating centroids until the centroids no longer move significantly or the maximum number of iterations is reached. This means the clusters have stabilized.

Convergence:

The algorithm stops when it converges, meaning the cluster assignments and centroid positions are stable, or other stopping criteria are met.

What does fitting achieve?

- The algorithm finds the optimal clusters based on minimizing the within-cluster variance (sum of squared distances between points and their cluster centroid).
- It organizes your data points into groups where points in the same cluster are similar to each other.

How to Fit the Model (Python Example)

```
# Assuming kmeans is already initialized and data is prepared
# Fit the KMeans model to the data
kmeans.fit(data)
```

This single command runs the entire clustering process on your dataset.

After fitting, the model contains information about the final cluster centroids and labels for each data point

Obtain the Results

After running the K-means algorithm on your dataset (fitting the model), the next step is to extract and interpret the results. These results give you insights into how the data was grouped.

Key outputs to obtain:

Cluster Centroids:

These are the coordinates of the center points of each cluster.

Each centroid represents the "average" position of all data points assigned to that cluster across all features.

Centroids help summarize the characteristics of each cluster and can be used to understand or label the groups.

Cluster Labels:

Each data point is assigned a label that indicates which cluster it belongs to.

Labels are usually integers ranging from 0 to K-1 (where K is the number of clusters).

These labels can be used to segment your data or for further analysis, like comparing cluster properties or visualization.

How to Extract Results in Python

Assuming you have a fitted KMeans model named kmeans:

```
# Get the coordinates of the cluster centroids
centroids = kmeans.cluster_centers_
# Get the cluster label for each data point
labels = kmeans.labels_
print("Centroids:\n", centroids)
print("Labels:", labels)
```

cluster_centers_ is a NumPy array where each row corresponds to a centroid's coordinates.

labels_ is an array with the same length as your dataset, indicating the cluster index assigned to each point.

Why Are These Results Important?

- Centroids give you a summary of each cluster's location in feature space, which is useful for interpreting cluster characteristics or making predictions about new data points.

- Labels allow you to group or filter your data based on cluster membership, enabling tasks like targeted marketing, anomaly detection, or data summarization.

(Optional) Visualize or Analyze the Clusters

Once you have clustered your data and obtained the cluster labels and centroids, the next step is to visualize or analyze these clusters. This helps you understand the structure of your data, assess the quality of clustering, and communicate insights effectively.

Why visualize or analyze clusters?

Interpretability: Visualization makes it easier to see how well your data points are grouped and how distinct the clusters are.

Validation: You can verify whether the clustering results meet your expectations or make sense for your specific problem.

Decision-making: Visual insights can guide business or research decisions, such as targeting specific customer segments or identifying anomalies.

Common Visualization Techniques

- **Scatter Plots (2D or 3D):**

Plot the data points with different colors or markers for each cluster label.

Plot cluster centroids to show the cluster centers visually.

Useful when your data has 2 or 3 features.

- **Pair Plots or Feature Plots:**

When you have more features, plot pairs of features colored by cluster label to see how clusters separate in different feature combinations.

Cluster Size Bar Chart:

Show the number of points in each cluster to understand cluster distribution.

- **Silhouette Plots or Other Cluster Quality Metrics:**

Use metrics and corresponding plots to evaluate how well-separated and compact clusters are.

Example: Visualizing Clusters Using Matplotlib (2D Data)

```
import matplotlib.pyplot as plt
# Assuming 'data', 'labels', and 'centroids' are available
# Assign colors based on cluster labels
colors = ['red' if label == 0 else 'blue' for label in labels]
# Plot data points colored by cluster
plt.scatter(data[:, 0], data[:, 1], c=colors, alpha=0.6, edgecolor='k')
# Plot centroids with a distinctive marker
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='yellow', marker='*')
plt.title('K-Means Clustering Visualization')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Analyzing Clusters

- Cluster Profiles: Calculate summary statistics (mean, median, range) for each cluster's features to understand their characteristics.
- Use Labels for Segmentation: Filter or group your dataset by cluster labels for targeted analysis or decision-making.
- Compare Clusters: Look for differences or similarities across clusters that may be meaningful for your application.

General Syntax Template (Python example)

```
# Step 1: Import libraries
from sklearn.cluster import KMeans
import numpy as np

# Step 2: Prepare dataset
data = np.array([[...], [...], ...])  # Your data points here
```

```
# Step 3: Initialize KMeans model
kmeans = KMeans(n_clusters=K,
random_state=seed_value)
# Step 4: Fit the model
kmeans.fit(data)
# Step 5: Get results
centroids = kmeans.cluster_centers_
labels = kmeans.labels_
# Step 6: (Optional) Visualize or analyze
# e.g., plot clusters, print labels, etc.
```

Concrete Example with Explanation

Imagine you have a small dataset of points in a two-dimensional space. These points represent some observations or measurements, such as locations, customer attributes, or any other features with two variables.

In this example, the dataset consists of six points: three clustered near coordinates around 1 on the x-axis, and three clustered near 10 on the x-a

Problem

- You want to automatically group these points into meaningful clusters based on their similarity. Specifically, you want to divide the points into two groups (clusters) where the points in the same group are closer to each other than to points in other groups.

- This task is common in many fields, such as customer segmentation, image analysis, or pattern recognition, where identifying natural groups helps in understanding or acting on the data.

Task

The task is to:

- Use the K-means clustering algorithm to partition the data into two clusters.
- Identify the centroids (center points) of these clusters.
- Assign each data point to one of the two clusters based on proximity to the centroids.
- Visualize the clusters and their centroids to see how the points are grouped.

The provided code accomplishes this by:

- Initializing the K-means model with 2 clusters.
- Fitting the model to the dataset to find cluster centers and assignments.
- Printing the centroids and cluster labels for each point.
- Plotting the points colored by cluster and marking the centroids for clear visualization.

Step 1: Import Necessary Libraries

```
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt
```

Explanation:

We import essential libraries:

numpy for handling numerical data and arrays,

KMeans from sklearn.cluster to perform the K-means clustering,

matplotlib.pyplot for visualizing the clusters and centroids.

Step 2: Prepare the Dataset

```
data = np.array([[1, 2], [1, 4], [1, 0],
                 [10, 2], [10, 4], [10, 0]])
```

Explanation:

Here, we define a small 2D dataset with six points. The points naturally form two groups: three points near (1, y) and three points near (10, y).

Step 3: Initialize the K-Means Model

```
kmeans = KMeans(n_clusters=2, random_state=0)
```

Explanation:

We initialize the K-means model specifying we want to create 2 clusters (n_clusters=2). Setting random_state=0 ensures that the centroid initialization is reproducible each time the code runs.

Step 4: Fit the Model to the Data

```
kmeans.fit(data)
```

Explanation:

This runs the K-means algorithm on the dataset. It iteratively assigns points to clusters and recalculates centroids until convergence is reached.

Step 5: Obtain Cluster Centroids and Labels

```
centroids = kmeans.cluster_centers_
labels = kmeans.labels_
print("Centroids:\n", centroids)
print("Labels:", labels)
```

Explanation:

cluster_centers_ gives the coordinates of the two cluster centroids found by the algorithm.

labels_ assigns each data point to a cluster (0 or 1).

Printing these helps you understand where the clusters are and which points belong to which cluster.

Step 6:Visualize Clusters and Centroids

```
colors = ['red' if label == 0 else 'blue' for label in labels]
plt.scatter(data[:, 0], data[:, 1], c=colors)
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='yellow',
marker='*')
plt.title('K-Means Clustering Example')
plt.show()
```

Explanation:

We assign colors to points based on their cluster labels red for cluster 0, blue for cluster 1.

We plot the data points with these colors to visually show cluster membership.

The centroids are plotted as large yellow stars to clearly mark the centers of each cluster.

The plot gives a visual understanding of how the algorithm grouped the points.